

PAYTM APPLICATION

Aasmaan Gupta IMT2021006, Mayank Sharma IMT2021086

OVERVIEW

The **PaytmPayment System** is a web-based application that enables users to perform various financial transactions securely and efficiently. This system provides features like user authentication, fund transfers between users, and account balance viewing. Additionally, it integrates **sentiment analysis** on transaction notes, offering dynamic UI and personalized feedback message based on the tone of the message.

GitHub Link

<https://github.com/Aasmaan007/paytm-spe2-/tree/tests>

What is Dev-Ops ?

DevOps is a collection of practices and principles designed to enhance communication and collaboration between development and operations teams, ultimately boosting the speed and quality of software delivery. By automating processes, adopting agile methodologies, and utilizing tools for continuous integration, delivery, and deployment, DevOps promotes a streamlined workflow. The goal is to break down organizational silos and foster collaboration, allowing for faster software delivery, fewer errors, and greater efficiency. Some key practices and principles of DevOps include:

-
- a) **Continuous Integration (CI):** Developers frequently integrate their code into a shared repository, triggering automated builds and tests.
 - b) **Continuous Delivery (CD):** Code changes are automatically built, tested, and deployed to production.
 - c) **Infrastructure as Code (IaC):** Infrastructure is defined and managed through code and version control, enabling versioning, testing, and automated deployment.
 - d) **Monitoring and Logging:** Real-time monitoring and logging allow teams to swiftly detect and resolve issues.
 - e) **Collaboration and Communication:** DevOps places a strong emphasis on communication and collaboration between developers, operations teams, and other stakeholders.

Why Dev-Ops ?

DevOps is crucial because it helps organizations deliver software faster, with fewer errors, and more efficiently. Here are some key reasons why DevOps is important:

- a) **Faster Time-to-Market:** DevOps allows organizations to release software more quickly, enabling them to better respond to changing market conditions and customer demands.
- b) **Improved Quality:** By automating processes and adopting agile methodologies, DevOps reduces errors in software releases, leading to higher quality software.
- c) **Enhanced Collaboration:** DevOps fosters collaboration and communication between development and operations teams, breaking down silos and improving overall organizational efficiency.
- d) **Scalability:** DevOps practices help organizations scale their software delivery processes to accommodate user growth, without compromising on quality or speed.
- e) **Better Security:** By integrating security into the development lifecycle, DevOps helps mitigate the risk of vulnerabilities and data breaches.

Application Features

- a) User Authentication
- b) Fund Transfers Between Users
- c) Account Balance Viewing

-
- d) User sentiment analysis on transaction notes
 - e) Personalized Feedback Message and Dynamic UI based on sentiment.

Project Technology stack

1. Frontend:

- **React.js:** Used for building a dynamic and responsive user interface.
- **React Router:** For navigating between different pages (e.g., SignUp, SignIn, Dashboard).
- **CSS/HTML:** For styling the frontend components and structuring the layout of the web application.

2. Backend:

- **Express.js:** A minimal and flexible Node.js web application framework
- **Node.js:** JavaScript runtime used for the backend development.

3. Storage:

- **MongoDB:** A NoSQL database used to store user information, account balances, and transaction data.
- **Mongoose:** ODM (Object Data Modeling) library for MongoDB and Node.js, used to define data models and interact with the database in an object-oriented manner.

4. Version Control System (VCS) / Source Control Management (SCM):

- **Git:** A distributed version control system used to track changes in the codebase.
- **GitHub:** A platform for hosting and sharing the project code, collaborating with others, and managing version control.

5. Testing:

- **Supertest:** A library used for testing HTTP APIs, especially for sending requests and asserting responses in the backend

6. Build:

- **npm (Node Package Manager):** Used for managing project dependencies, running scripts, and building the frontend and backend of the application.

7. Continuous Integration (CI) Tool:

- **Jenkins:** An open-source automation server used to build, test, and deploy the project. Jenkins automates the CI/CD pipeline, ensuring that code changes are tested and deployed consistently.

8. Continuous Deployment (CD) Tools:

-
- **Docker:** A tool used to containerize the frontend and backend applications. This ensures that the application runs consistently across different environments.
 - **Ansible:** An automation tool used for deploying the application to servers. It manages configurations and automates the deployment process using playbooks.

9 Logging

- **Winston:** A versatile logging library used for logging different levels of application activity. It allows the creation of log files for specific types of events or errors, which helps in tracking, debugging, and maintaining an application's behavior over time.
- **Combined Log:** Captures all logs (info, warn, error) in one file for a comprehensive overview of system activity.
- **Info Log:** Logs informational messages, tracking successful operations and normal application flow.
- **Warn Log:** Records warnings for potential issues that don't cause failures but need attention.
- **Error Log:** Logs critical errors and failures, highlighting issues that need immediate resolution.

DevOps Tools

1. **Source Control Management (SCM):**
 - **Git:** A distributed version control system used to track changes in the project's source code.
 - **GitHub:** A web-based platform for hosting Git repositories, version control, collaboration, and code sharing.
2. **Continuous Integration / Continuous Deployment (CI/CD) Pipeline:**
 - **Jenkins:** Jenkins automates the process of building, testing, and deploying the application, ensuring that code changes are automatically integrated and tested.
3. **Containerization:**
 - **Docker:** A tool used to package and run applications in containers.
 - **DockerHub:** A cloud-based registry service used for sharing and storing Docker images.
4. **Container Orchestration:**
 - **Docker Compose:** A tool used for defining and running multi-container Docker applications. Docker Compose allows you to define the services (e.g., frontend,

backend, database) in a `docker-compose.yml` file and easily start and manage them together.

5. Configuration Management:

- **Ansible:** A powerful automation tool used for configuration management and application deployment. In this project, Ansible is used to deploy the application and manage server configurations. It automates the setup of environments, ensuring that the application is consistently deployed with the correct configurations across different environments.

6. Monitoring ELK Stack (Elasticsearch, Logstash, Kibana):

- **Elasticsearch:** A search and analytics engine used for storing and indexing log data.
- **Logstash:** A tool for managing and processing logs. It helps to collect logs from various sources (e.g., application, system) and send them to Elasticsearch for indexing.
- **Kibana:** A data visualization platform that integrates with Elasticsearch. It allows you to create interactive dashboards and visualize the data (logs, metrics) stored in Elasticsearch, enabling real-time monitoring of the application's performance.
- **Filebeat:** A lightweight log shipper used to send log files from the application directly to Elasticsearch for indexing. Filebeat reads and forwards log data from the application's log files (e.g., `combined.log`, `info.log`, `warn.log`) to Elasticsearch, where it can be visualized using Kibana. This enables seamless log management and monitoring without the need for manual intervention.

7. Testing

- In this testing setup, the `supertest` library was utilized to perform integration tests on the user routes of the application. The tests focus on verifying the functionality of the `signup` and `signin` API endpoints.
- The `supertest` package allows for HTTP requests to be made directly to the application without requiring an actual server to be running. This helps simulate the behavior of real-world HTTP requests and responses. The tests ensure that the expected status codes and response messages are returned for successful and error scenarios.
- For instance, the first test checks if the `signup` endpoint correctly creates a new user and returns a success message along with a token. The second test ensures that a user can successfully log in using their credentials, while the third test verifies that the system handles duplicate email registrations by returning a conflict error. Additionally, cleanup functions are employed to maintain a clean state between tests, such as removing test users from the database after each test case. This structured approach ensures the reliability and correctness of the user routes while preventing side effects between tests.

All tests are executed using the `npm test` command

```

1 const request = require('supertest');
2 const app = require('../index'); // Import the app from your index.js
3 const { User } = require('../db.js');
4
5 describe('User Routes', () => {
6   beforeAll(async () => {
7     // Add any setup logic if needed, such as connecting to the database
8   });
9
10  afterAll(async () => {
11    // Cleanup logic, such as closing database connections
12  });
13
14  afterEach(async () => {
15    // Cleanup logic for test isolation, such as removing test users
16    await User.deleteMany({});
17  });
18
19  test('POST /api/v1/user/signup - should create a new user', async () => {
20    const response = await request(app)
21      .post('/api/v1/user/signup')
22      .send({
23        username: 'testuser@example.com',
24        firstName: 'Test',
25        lastName: 'User',
26        password: 'password123',
27      });
28
29    expect(response.status).toBe(201);
30    expect(response.body).toHaveProperty('message', 'User created successfully');
31    expect(response.body).toHaveProperty('token');
32  });
33
34  // Additional tests for GET, PUT, and DELETE endpoints
35});

```

user.test.js

Project Setup

These are the steps that we followed to setup our project:

1. Installing Jenkins:

- `wget -q -O - https://pkg.jenkins.io/debian-stable/jenkins.io.key | sudo`
- `apt-key add-`
- `sudo sh -c 'echo deb http://pkg.jenkins.io/debian-stable binary/ >`
- `/etc/apt/sources.list.d/jenkins.list'`
- `sudo apt install ca-certificates`
- `sudo apt-get update`
- `sudo apt-get install jenkins`

2. Admin Password for Jenkins:

- After installing Jenkins open a tab on web browser and load
- `http://localhost://8080`.
- Follow the path and use cat (`sudo cat`
- `/var/lib/jenkins/secrets/initialAdminPassword`) in order to get the password
- for the admin user.
- Store this password to access jenkins later on.

4. Setting up dockerhub:

- Sign up on the dockerhub platform creating an account with your username
- and password.
- Create a new repository which is where the docker images will be pushed
- and pulled from
- Go to Jenkins -> Manage Jenkins -> Manage Credentials -> {global} -> Add new
- Credential -> Add the username and password with the credential name you
- want to associate with it. This credential can be later used for pushing and
- pulling the docker images

5. Installing required plugins - Jenkins:

- Login to jenkins using the admin credentials.

-
- Click on Manage Jenkins -> Manage Plugins -> Available Plugins.
 - Install Plugins for: Git, Docker - docker api, docker commons plugin, docker pipeline, docker plugin, docker build setup, Ansible etc.

6. Setting up Jenkins:

- Click on New item -> Give a name to your jenkins pipeline -> Select Pipeline
- [for style of project] -> Save
- Select 'Github Project' and copy paste the project url into the textbox below
- it.
- From the 'Build Triggers' menu select 'Github hook trigger for GitScm Polling'
- Setting up Github Webhook:
- Go to Manage Jenkins -> Configure System -> and set Jenkins URL to ngrok url(copied).

7. Pipeline stages: Check Pipeline Code from our github repo.

a) Pipeline Overview

The pipeline is defined using the Declarative Pipeline syntax, which provides a structured way to define the pipeline. It specifies the environment variables, stages, and steps involved in the process. The environment variables are:

- **backend**: The Docker image name for the backend service.
- **frontend**: The Docker image name for the frontend service.
- **VAULT_PASSWORD**: The password for Ansible Vault, used for secure handling of sensitive data during deployment.

b) Environment Block

The **environment block** sets up variables that are used throughout the pipeline. These variables simplify the code by avoiding hardcoding values. For example:

- The **backend** and **frontend** variables define the names of the Docker images that will be built and pushed.
- The **VAULT_PASSWORD** variable is used to handle sensitive information (like a password) securely, especially for Ansible Vault during deployment.

```
pipeline {
  environment {
    backend = 'backend' // Backend Docker image name
    frontend = 'frontend' // Frontend Docker image name
    VAULT_PASSWORD = 'aasmaan' // Jenkins secret for the Ansible vault password
  }
}
```

c) Stages Breakdown

The **stages** section defines various sequential steps that the pipeline will execute.

Below are the individual stages:

- Clone Repository, clones the **GitHub repository** where the project's code is stored.

```
stages {
  stage('Clone Repository') {
    steps {
      echo 'Cloning the Git repository...'
      git branch: 'master', url: 'https://github.com/Aasmaan007/paytm-spe2-.git'
    }
  }
}
```

- Running tests , In this stage, automated tests are executed to ensure the integrity and functionality of the backend application. First, the necessary dependencies are installed using **npm install**. Then, the **npm test** command is run to trigger the test suite, which verifies that the API endpoints, including user signup and login, work as expected. The tests also check for edge cases and errors to ensure that the application handles various scenarios correctly.


```

stage('Testing') {
  steps {
    echo 'Running tests...'
    script {
      // Navigate to the backend directory
      dir('backend') {
        // Install dependencies
        sh 'npm install'

        // Run tests
        sh 'npm test'
      }
    }
  }
}

```

- Stop and Remove Containers, This stage stops and removes any running or stopped Docker containers before building new ones. This ensures that no old containers interfere with the new build and deployment.

```

stage('Stop and Remove Containers') {
  steps {
    echo 'Stopping and removing all Docker containers...'
    script {
      def containerIds = sh(script: "sudo docker ps -aq", returnStdout: true).trim()
      if (containerIds) {
        echo "Stopping and removing containers: ${containerIds}"
        sh "sudo docker stop \$(sudo docker ps -aq)"
        sh "sudo docker rm \$(sudo docker ps -aq)"
      } else {
        echo 'No containers found. Skipping stop and remove steps.'
      }
    }
  }
}

```

- Build Docker Images , This stage builds Docker images for both the **backend** and **frontend** services.

```

stage('Build Docker Images') {
  steps {
    echo 'Building Docker images...'
    script {
      backend_image = docker.build("aasmaan1/backend:latest", "./backend")
      frontend_image = docker.build("aasmaan1/frontend:latest", "./frontend")
    }
  }
}

```

- Push Docker Images to Docker Hub, This stage pushes the built Docker images (both backend and frontend) to **Docker Hub** for storage and easy deployment across environments.

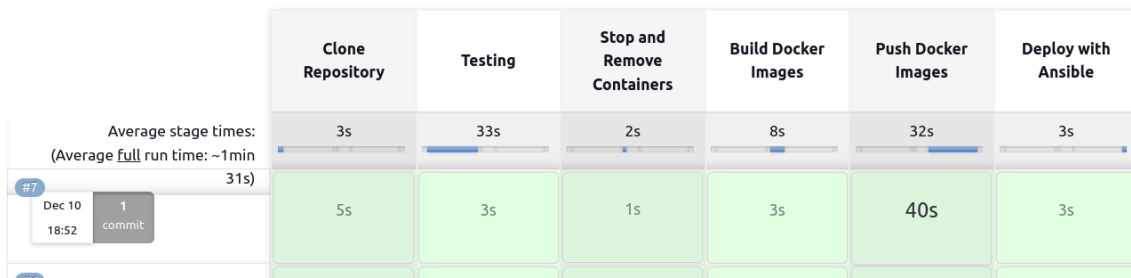
```
stage('Push Docker Images') {
    steps {
        echo 'Pushing Docker images to Docker Hub...'
        script {
            docker.withRegistry('', 'DockerHubCred') {
                backend_image.push()
                frontend_image.push()
            }
        }
    }
}
```

- Deploy with Ansible, This stage handles the **deployment** of the application using **Ansible**. It runs an **Ansible playbook** to configure the target server and deploy the app. It makes uses of Ansible Vaults to securely inject MongoURL into .env file of backend and also uses Ansible Roles to implement Modular design in our code.

```
stage('Deploy with Ansible') {
    steps {
        echo 'Running Ansible playbook for deployment...'
        script {
            writeFile file: '/tmp/vault_password.txt', text: "${env.VAULT_PASSWORD}"
        }
        ansiblePlaybook(
            playbook: 'Deployment/deploy.yml',
            inventory: 'Deployment/inventory',
            extras: "--vault-password-file /tmp/vault_password.txt",
            credentialsId: 'localhost',
            disableHostKeyChecking: true
        )
    }
}
```

This Jenkins pipeline script automates the entire CI/CD process for the **Paytm-Application** . From **repository cloning** to **Docker image building**, **test execution**, and **deployment using Ansible**, every step is automated.

Stage View



Successful Execution of Pipeline

8. Container Creation and Orchestration :

a) Backend Dockerfile (Containerization for Backend)

This Dockerfile is used to create a Docker image for the Node.js backend of the application. It uses a multi-step process to ensure that the backend is packaged correctly for deployment.

```
Dockerfile backend X Dockerfile frontend
backend > Dockerfile > ...
1  # Use Node.js LTS base image
2  FROM node:16
3
4  # Set the working directory inside the container
5  WORKDIR /app
6
7  # Copy package.json and package-lock.json to install dependencies
8  COPY package*.json ./
9
10 # Install dependencies
11 RUN npm install
12
13 # Copy the rest of the application files
14 COPY . .
15
16 # Expose the backend port (3000)
17 EXPOSE 3000
18
19 # Start the backend server
20 CMD ["npm", "start"]
21 |
```

1. **Base Image:** Uses the official **Node.js 16** LTS image for the backend.
2. **Working Directory:** Sets **/app** as the directory where all app files will reside inside the container.
3. **Install Dependencies:** Copies **package.json** and **package-lock.json** and runs **npm install** to install dependencies.

4. **Copy Application Code:** Copies the rest of the application files into the container.
5. **Expose Port:** Exposes **port 3000** for the backend server to be accessed externally.
6. **Start Backend Server:** Runs the **npm start** command to start the Node.js server.

b) Frontend Dockerfile (Containerization for Frontend)

Below dockerfile is used to create a Docker image for the React.js frontend of the application.

```
frontend >  Dockerfile > ...  
1      # Stage 1: Build the React app  
2      FROM node:16 as build  
3  
4      # Set the working directory  
5      WORKDIR /app  
6  
7  
8      # Copy package.json and package-lock.json  
9      COPY package*.json ./  
10  
11     # Install dependencies  
12     RUN npm install  
13  
14     # Copy all project files (including .env if necessary)  
15     COPY . .  
16  
17     # Set the environment variable for build time  
18     # This will allow React to use the correct backend URL during the build  
19  
20     # Build the app (using the .env file copied earlier)  
21     RUN npm run build  
22  
23     # Stage 2: Serve with Nginx  
24     FROM nginx:stable-alpine  
25  
26     # Copy build output to Nginx html directory  
27     COPY --from=build /app/dist /usr/share/nginx/html  
28  
29     # Expose the port (80 for Nginx)  
30     EXPOSE 80  
31  
32     # Start Nginx  
33     CMD ["nginx", "-g", "daemon off;"]  
34
```

1. Stage 1 - Build React App:

- Uses **Node.js 16** image to build the React app.
- Installs dependencies and builds the React application (outputs to **dist** folder).

2. Stage 2 - Serve with Nginx:

- Uses **Nginx Alpine** to serve the production-ready static files.
- Copies the built files from Stage 1 into Nginx's default directory.
- Exposes **port 80** for the frontend to be served via Nginx.

c) Docker Compose (Container Orchestration)

Docker Compose is used to manage multi-container Docker applications by defining a set of services in a single `docker-compose.yml` file. In our case, the file orchestrates the deployment of the backend and frontend containers.

- The `image` field in the file specifies which pre-built Docker image should be used for each container. For instance, `aasmaan1/backend:latest` pulls the latest backend image from Docker Hub, which was previously built and pushed during the Jenkins pipeline process. Similarly, the frontend container uses `aasmaan1/frontend:latest` to pull the frontend image.
- The `ports` field is used to map ports on the host machine to ports inside the container. For example, `"3000:3000"` maps the host machine's port 3000 to the container's port 3000 for the backend service, allowing external access to the backend application. Likewise, `"5173:80"` maps port 5173 on the host to port 80 inside the frontend container, making the frontend application accessible on the host machine.

9. Configuration Management (Ansible)

Ansible is an open-source automation tool used for configuration management, application deployment, and task automation. In our project, Ansible is used to automate the deployment of Docker containers for the backend and frontend applications, ensuring that the deployment process is consistent, repeatable, and efficient.

```
1  ---
2  - name: Deploy Docker Containers
3    hosts: localhost
4    vars_files:
5      - secrets.yml
6
7    roles:
8      - backend
9      - docker
```

Deploy.yml

The `deploy.yml` playbook defines the steps required to deploy the project. It does so by using different roles and tasks. Ansible uses **Roles** to organize tasks into reusable components. This allows for better organization, maintainability, and reusability of tasks across different

environments. In our project, we use the **backend** and **docker** roles to modularize the deployment process.

Ansible Vault is used to securely store sensitive information, such as the MongoDB URL (**MONGO_URI**). In our playbook, we use **vars_files** to include **secrets.yml**, which is where sensitive data is stored in an encrypted format. This data is then injected safely into the backend **.env** file, ensuring that secrets are not exposed in plain text during deployment.

```
Deployment > ! secrets.yml
1  $ANSIBLE_VAULT;1.1;AES256
2  65346333356132626234393331373764393031306435633132323532396332613061646662303062
3  3432623635366134663238373832306166373431306232630a346130663161383538343438393062
4  34366361393962653239366536393166303035346465363038633966666538376439616132343732
5  3831623561356634370a333938613435343338326335613839656661376533613037373638656230
6  63393330623338613238633534343137636235653637316437643562306233353962653463356330
7  31333638303435303566356635616236376263643962613862633365356663303833653330386164
8  35393936373766626462636165323331663363393632343965346331393735333565356434326563
9  65623938353665636364396462616630333464653036646136636337643539323866653639396562
10 35643237363833646331363530616133326164646435623632336465373430613736363563626531
11 37393561663030383661653439656530393361323634366636333939396532313232636637633164
12 31303830613165353564316232653163303766323164303834643236343439393139653361383366
13 32643665343863646434
```

Encrypted secrets.yml

```
1  ---
2  # Backend role: tasks related to the backend service
3
4  - name: Create backend environment file dynamically
5    shell: echo "MONGO_URI={{ db_url }}" > ../backend/.env
6    args:
7      chdir: "{{ playbook_dir }}" # Ensure correct directory is used to run the shell command
8      become: true # Optionally run as root if needed
9
```

roles/backend/tasks/main.yml

Backend Role: This role handles tasks related to the backend service. For example, it creates a dynamic environment file (**.env**) in the backend directory. The environment file stores sensitive information such as the MongoDB URI (**MONGO_URI**), which is used to connect to the database. This ensures that the backend has the correct configuration when it starts.

```

Deployment > roles > docker > tasks > ! main.yml
2  # Docker role: tasks related to starting Docker containers
3
4  - name: Start Docker containers (using images from DockerHub)
5    shell: docker-compose up -d
6    args:
7      chdir: "{{ playbook_dir }}" # Use playbook directory for Docker Compose
8    become: true # Optionally run as root if needed
9

```

roles/docker/tasks/main.yml

Docker Role: The Docker role manages the Docker container orchestration. It uses the `docker-compose up -d` command to start the containers defined in the `docker-compose.yml` file. By using this role, Ansible ensures that the necessary containers (frontend and backend) are launched and running in detached mode.

10. Monitoring and Logging: ELK Stack (Elasticsearch, Logstash, and Kibana)

In our project, Winston was utilized to handle logging across four different types of log files—`error.log`, `warn.log`, `info.log`, and `combined.log`. These log files captured important system events and were classified by their severity levels: Error, Warn, Info, and Combined. The logs were generated using the Winston logger integrated into both the backend API and other services, ensuring critical issues and general activity were logged appropriately.

```

10  // workgit
11  const logger = winston.createLogger({
12    level: 'info',
13    format: winston.format.combine(
14      winston.format.timestamp(),
15      winston.format.json()
16    ),
17    transports: [
18      new winston.transports.File({ filename: 'logs/error.log', level: 'error' }),
19      new winston.transports.File({ filename: 'logs/warn.log', level: 'warn' }),
20      new winston.transports.File({ filename: 'logs/info.log', level: 'info' }),
21      new winston.transports.File({ filename: 'logs/combined.log' })
22    ]
23  });
24

```

winston library to generate logs

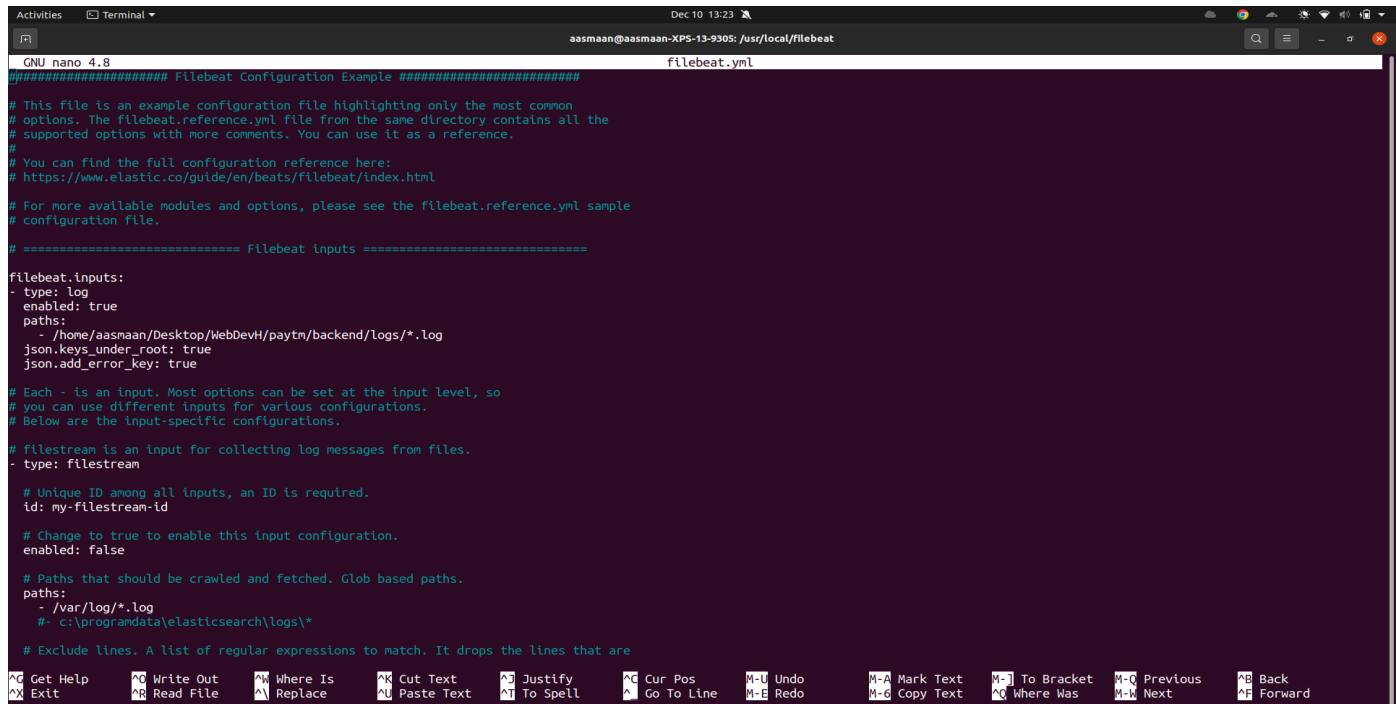
```

1  {"level":"info","message":"User retrieved successfully","timestamp":"2024-11-26T11:23:25.765Z","userId":"673
2  {"filter":"","level":"info","message":"Bulk user retrieval successful","timestamp":"2024-11-26T11:23:25.769Z
3  {"level":"info","message":"User retrieved successfully","timestamp":"2024-11-26T11:23:25.796Z","userId":"673
4  {"filter":"","level":"info","message":"Bulk user retrieval successful","timestamp":"2024-11-26T11:23:25.798Z
5  {"balance":9950.299960684806,"level":"info","message":"Account balance retrieved successfully","timestamp":
6  {"balance":9950.299960684806,"level":"info","message":"Account balance retrieved successfully","timestamp":
7  {"email":"aasmaan@gmail.com","level":"info","message":"User signed in successfully","timestamp":"2024-11-26T
8  {"filter":"","level":"info","message":"Bulk user retrieval successful","timestamp":"2024-11-26T11:23:56.548Z
9  {"level":"info","message":"User retrieved successfully","timestamp":"2024-11-26T11:23:56.554Z","userId":"673
10 {"balance":9950.299960684806,"level":"info","message":"Account balance retrieved successfully","timestamp":
11 {"filter":"","level":"info","message":"Bulk user retrieval successful","timestamp":"2024-11-26T11:23:56.573Z
12 {"level":"info","message":"User retrieved successfully","timestamp":"2024-11-26T11:23:56.586Z","userId":"673
13 {"balance":9950.299960684806,"level":"info","message":"Account balance retrieved successfully","timestamp":
14 {"amount":12,"fromUserId":"67398927ebc469e291ccda2d","level":"info","message":"Transfer successful","times

```

combined.log

To effectively monitor these logs, Filebeat was set up to ship the generated logs from the system to Elasticsearch. The `filebeat.yml` configuration file was used to define the paths for the log files as the source and Elasticsearch as the destination. This setup ensured that the logs were securely transmitted and indexed for further analysis.



```

filebeat.yml

# This file is an example configuration file highlighting only the most common
# options. The filebeat.reference.yml file from the same directory contains all the
# supported options with more comments. You can use it as a reference.
#
# You can find the full configuration reference here:
# https://www.elastic.co/guide/en/beats/filebeat/index.html
#
# For more available modules and options, please see the filebeat.reference.yml sample
# configuration file.
#
# ===== Filebeat inputs =====

filebeat.inputs:
- type: log
  enabled: true
  paths:
    - /home/aasmaan/Desktop/WebDevH/paytm/backend/logs/*.log
  json.keys_under_root: true
  json.add_error_key: true

# Each - is an input. Most options can be set at the input level, so
# you can use different inputs for various configurations.
# Below are the input-specific configurations.

# filestream is an input for collecting log messages from files.
- type: filestream

  # Unique ID among all inputs, an ID is required.
  id: my-filestream-id

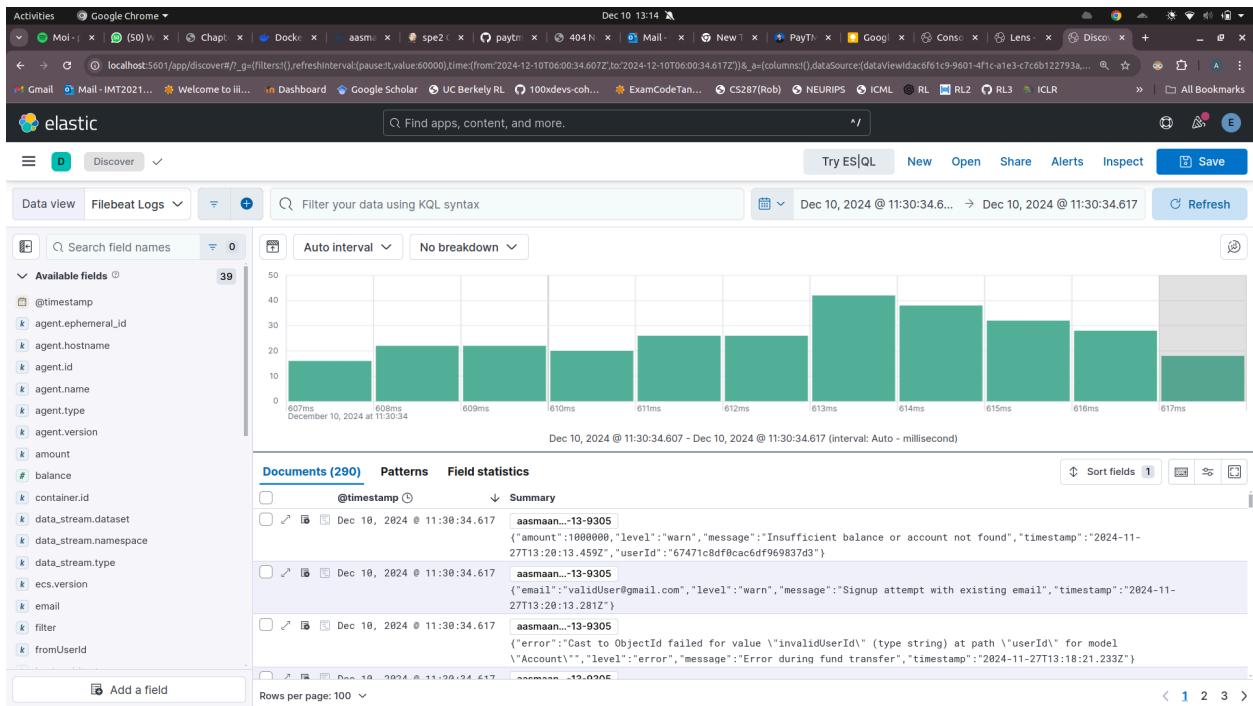
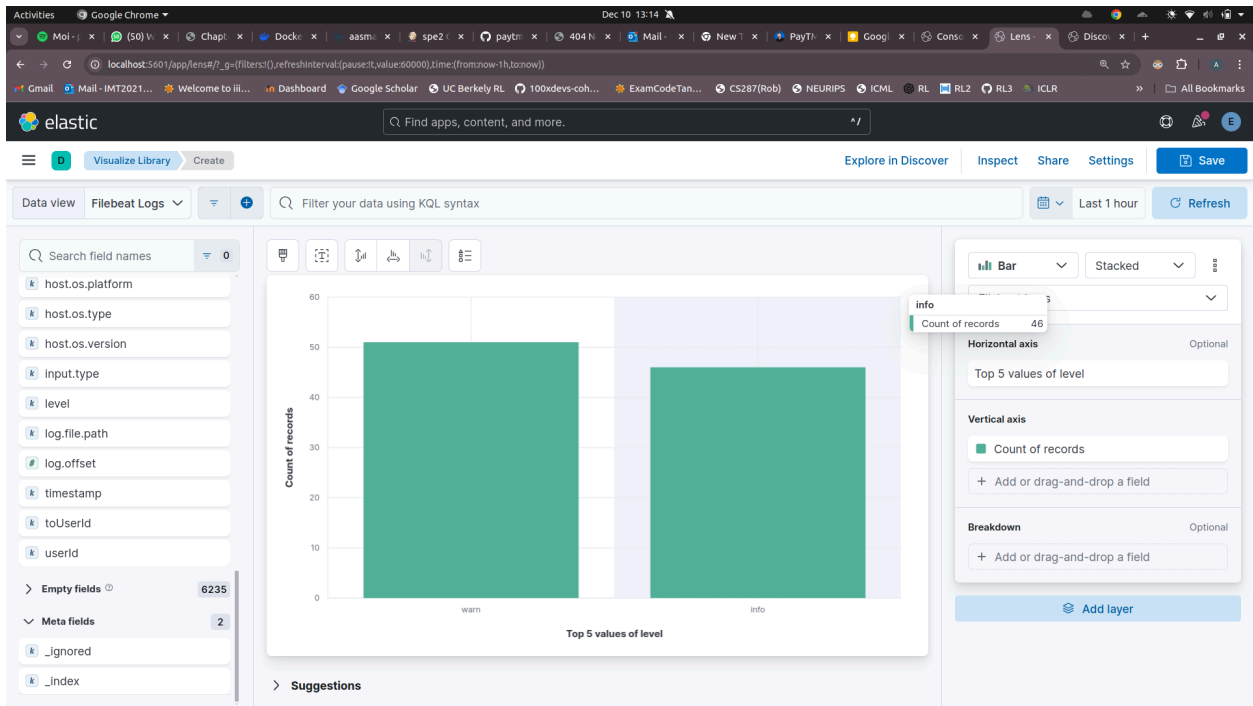
  # Change to true to enable this input configuration.
  enabled: false

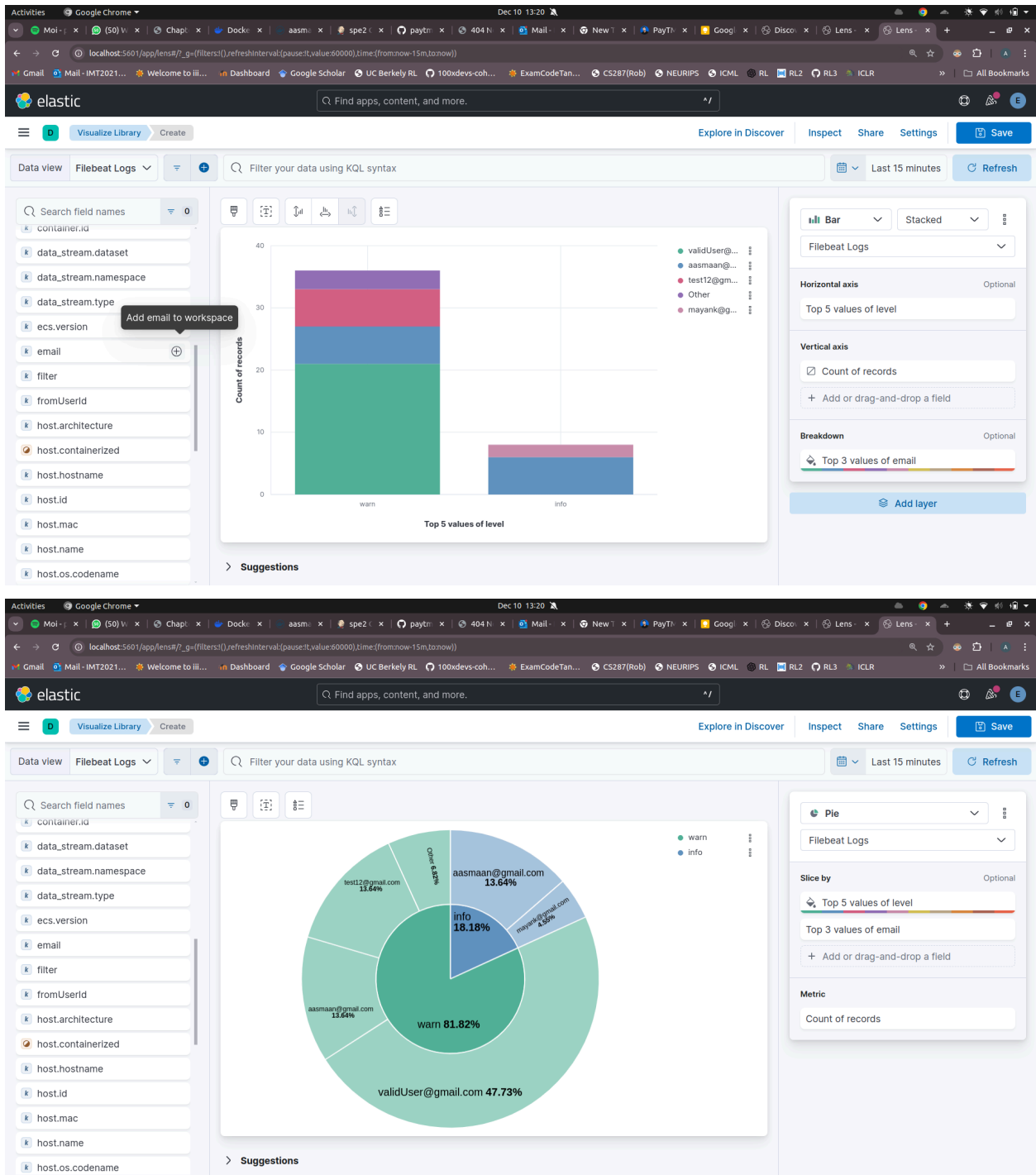
  # Paths that should be crawled and fetched. Glob based paths.
  paths:
    - /var/log/*.log
    #- c:\programdata\elasticsearch\logs\*

  # Exclude lines. A list of regular expressions to match. It drops the lines that are

```

Configuring filebeat.yml





Visualisations in Kibana

11. Advanced Features: Usage of Vault and Roles in Ansible

We leveraged Ansible Roles and Ansible Vault to streamline the deployment process and securely manage sensitive data.

Roles In ansible

Ansible Roles allow us to organize tasks into reusable, modular components. This modularization makes the deployment process cleaner, more scalable, and easier to manage.

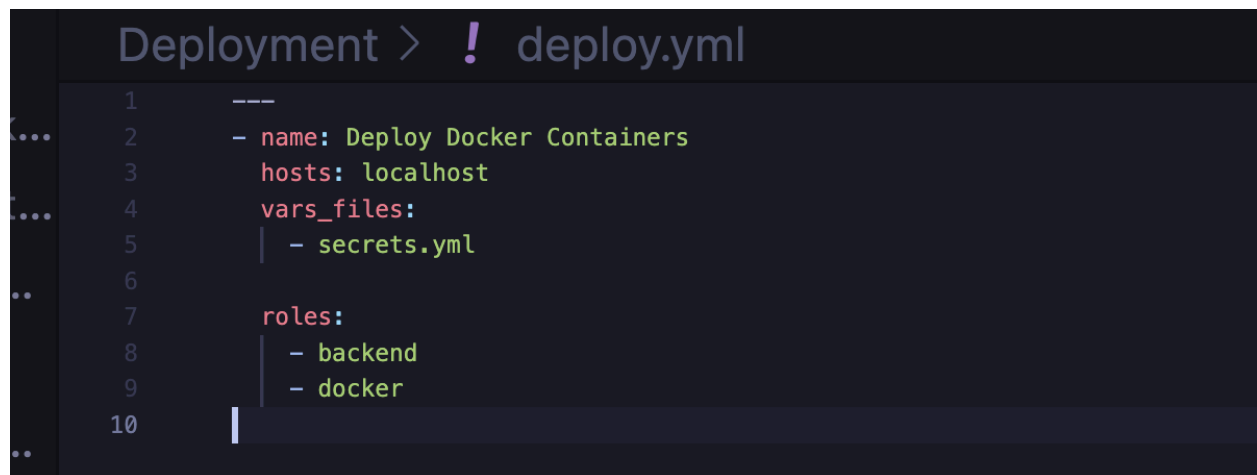
In our project, we defined two key roles:

- a) **backend**: Responsible for setting up the backend service.
- b) **docker**: Responsible for managing Docker-related tasks, such as pulling Docker images and starting containers.

These roles are defined inside the **roles/** folder, which helps us structure the playbook logically and makes it easy to reuse roles across different environments.

How Roles Are Used in **deploy.yml**:

The **deploy.yml** playbook is responsible for deploying the application and configuring the services. It calls both the **backend** and **docker** roles to manage different parts of the deployment process.



```
1  ---
2  - name: Deploy Docker Containers
3    hosts: localhost
4    vars_files:
5      - secrets.yml
6
7    roles:
8      - backend
9      - docker
10
```

vars_files: The **secrets.yml** file is referenced here to load sensitive data securely.

roles: The playbook calls both the **backend** and **docker** roles to handle specific tasks:

- The **backend** role configures the backend environment.
- The **docker** role manages Docker containers.

Roles in Action:

1) Backend role : The **backend** role sets up the backend environment by creating the **.env** file dynamically. It writes the MongoDB URL (passed as **db_url**) into the **.env** file in the backend directory. This task is executed from the playbook directory, ensuring the correct path. Elevated privileges are used to ensure proper permissions when writing the file.

```
Deployment > roles > backend > tasks > ! main.yml
1  ---
2  # Backend role: tasks related to the backend service
3
4  - name: Create backend environment file dynamically
5    shell: echo "MONGO_URI={{ db_url }}" > ../backend/.env
6    args:
7      chdir: "{{ playbook_dir }}" # Ensure correct directory is used to run the shell command
8      become: true # Optionally run as root if needed
9
```

2) Docker role : The **docker** role starts the Docker containers using **Docker Compose**. The **docker-compose up -d** command runs the containers in detached mode (in the background). This task is also executed from the **playbook directory**, ensuring that the **docker-compose.yml** file is used correctly. Elevated privileges are used to allow proper execution of Docker commands.

```
Deployment > roles > docker > tasks > ! main.yml
1  ---
2  # Docker role: tasks related to starting Docker containers
3
4  - name: Start Docker containers (using images from DockerHub)
5    shell: docker-compose up -d
6    args:
7      chdir: "{{ playbook_dir }}" # Use playbook directory for Docker Compose
8      become: true # Optionally run as root if needed
9
```

Vault Integration in Ansible:

Ansible Vault is used to securely manage sensitive data, such as passwords, API keys, and database credentials, by encrypting them. In this project, **Ansible Vault** stores the MongoDB

connection URL in an encrypted format, ensuring that sensitive data is securely handled during the deployment process.

Implementation of Vault

- **secrets.yml**: This file contains the MongoDB connection URL and is stored in an encrypted format using **Ansible Vault**. The contents of this file are used in the playbooks during deployment to retrieve sensitive configuration values securely. Password for decrypting this secrets.yml is specified in the Jenkins pipeline.

```
jenkinsfile
pipeline {
    environment {
        backend = 'backend' // Backend Docker image name
        frontend = 'frontend' // Frontend Docker image name
        VAULT_PASSWORD = 'aasmaan' // Jenkins secret for the Ansible vault password
    }
    agent any
```

Password for secrets.yml being specified in pipeline

```
script {
    // Create temporary file for vault password
    writeFile file: '/tmp/vault_password.txt', text: "${env.VAULT_PASSWORD}"
}
ansiblePlaybook(
    playbook: 'Deployment/deploy.yml',
    inventory: 'Deployment/inventory',
    extras: "--vault-password-file /tmp/vault_password.txt", // Use the temporary password :
    credentialsId: 'localhost'
```

VAULT_PASSWORD being used in final stage.

Usage in Playbooks

- The **secrets.yml** file is referenced in the **deploy.yml** playbook to access the MongoDB URL during deployment.
- The Vault password is stored separately and referenced during playbook execution to decrypt the file and inject the values into the environment.

12. . Innovation

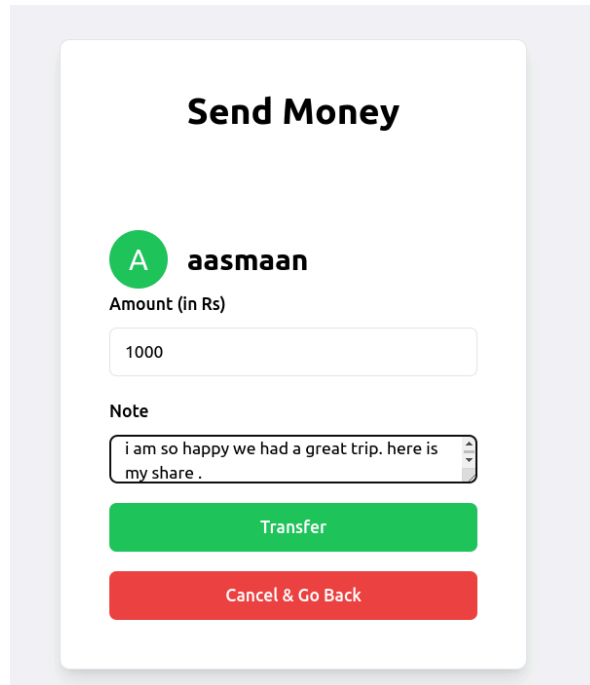
The Innovative feature implemented in the Paytm-Social Digital Payment System is the integration of sentiment analysis for transaction notes.

We used an external library, **Sentiment.js**, to analyze the sentiment of the transaction notes written by users. Based on the sentiment score, the system provides personalized

feedback and dynamic UI, encouraging more positive interactions and improving user engagement.

How it Works:

- **Feedback Messages:** A set of predefined messages are mapped to different sentiments.
 - **Positive:** "Thank you for spreading positivity!"
 - **Negative:** "Would you like to reconsider your message for a better experience?"
 - **Neutral:** "Neutral messages are okay, but adding a personal touch could make it special!"
- **Sentiment Analysis Function:** The function `analyzeSentiment` uses the `sentiment.analyze(text)` method to analyze the sentiment of the provided text. Based on the score:
 - If the score is positive, it returns a Positive sentiment with corresponding message and UI.



The image shows a mobile app interface for sending money. At the top, the title "Send Money" is displayed in bold black text. Below the title, there is a green circular profile picture with the letter "A" and the name "aasmaan" in bold black text. Underneath the name, the label "Amount (in Rs)" is shown. A text input field contains the number "1000". Below the input field, the label "Note" is displayed. A text area contains the message "i am so happy we had a great trip. here is my share .". At the bottom, there are two buttons: a green "Transfer" button and a red "Cancel & Go Back" button.

Transfer successful

Sentiment: Positive

**Feedback: Thank you
for spreading positivity!**

Redirecting to Dashboard in 5 seconds.

- If the score is negative, it returns a Negative sentiment with feedback encouraging reconsideration and corresponding UI.

Send Money



aasmaan

Amount (in Rs)

500

Note

it is really irritating that you are asking to
split for auto i am really disappointed

Transfer

Cancel & Go Back

Transfer successful

Sentiment: Negative

**Feedback: Would you
like to reconsider your
message for a better
experience?**

Redirecting to Dashboard in 5 seconds.

- If the score is neutral, it encourages users to add a more personal touch.